

Contents

guide_Vision-R1: Incentivizing Reasoning Capability in Multimodal Large Language

Models	1
0. 这篇导读怎么读	2
1. 历史语境: 为什么这篇论文会出现	2
2. 论文主张和贡献	3
3. 整体方法图: Fig.1 的真正含义	3
4. Modality bridging: Fig.2 的数据生成机制	4
5. Vision-R1-cold 数据集是什么	5
6. Cold-start initialization 为什么既必要又不充分	5
7. Overthinking optimization problem	6
8. PTST: Progressive Thinking Suppression Training	6
9. GRPO 公式怎么读	7
10. HFRRF: hard formatting result reward	8
11. 网络和信息流的张量图	8
12. Table 1: 主结果说明了什么	9
13. Table 2: Vision-R1-cold 数据质量证据	10
14. Table 3: 三个组件缺一不可	10
15. Table 5: PTST 为什么是两阶段	11
16. Table 6: cold-start 仍然是前提	11
17. Appendix A: 复现条件	12
18. 本仓库代码怎么对应论文	13
19. 代码样例: 用 toy 数据理解 modality bridging	13
20. 代码样例: GRPO 和 PTST 的最小数值图	14
21. 新手容易误读的地方	14
22. 局限性和你该保持的怀疑	15
23. 对今天多模态 agent 的意义	15
24. 怎样用 AI agent 学这篇论文, 才能真的进脑袋	15
25. 30-60 分钟本地练习	16
26. 闭卷掌握检查	17
27. 最后一口气总结	17

guide_Vision-R1: Incentivizing Reasoning Capability in Multimodal Large Language Models

manual-deep-guide: true

原论文: Vision-R1: Incentivizing Reasoning Capability in Multimodal Large Language Models

本地原文 PDF: learning/multimodal-agent/paper/01_vision_r1.pdf

作者: Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Xu Tang, Yao Hu, Shaohui Lin

版本: arXiv v4, 2026-02-28, ICLR 2026 conference paper

本地机制代码: learning/multimodal-agent/src/vision_r1_original_minimal.py

0. 这篇导读怎么读

这篇导读的目标不是把 Vision-R1 压缩成几条结论, 而是让你不打开论文也能抓住论文自己的内容: 当时为什么需要它, 它如何把 DeepSeek-R1 式强化学习 迁移到多模态模型, 为什么只做 RL 不够, 为什么 cold-start 之后还要 PTST, 以及实验表格到底支持了哪些 claim。

你需要带着四个问题读:

- DeepSeek-R1 的故事到了多模态场景为什么卡住了。
- Vision-R1-cold 数据集为什么要经过 modality bridging。
- GRPO 的 reward 为什么从 soft 加权变成 hard formatting result reward。
- PTST 为什么先压缩思考长度, 再逐步放松长度。

这篇论文的核心不是单个公式, 而是一条训练路线:

```
text-only R1 has strong reasoning
|
| problem: it cannot directly see images
v
MLLM converts image-question pairs into detailed text
|
v
DeepSeek-R1 generates complex CoT from bridged text
|
v
SFT cold-start teaches MLLM the complex reasoning style
|
v
GRPO plus PTST turns the style into more correct reasoning
```

一句话概括:

Vision-R1 先用 modality bridging 借 DeepSeek-R1 生成 200K 多模态复杂 CoT 作为 cold-start, 再用 GRPO 和 PTST 在多模态数学任务上强化正确且可控的推理轨迹。

1. 历史语境: 为什么这篇论文会出现

2024-2025 年前后, LLM reasoning 的主线出现了明显变化。早期大家靠 CoT prompting 或人工格式化步骤让模型看起来会推理, 但这些方法常常只是把答案拆成若干句。OpenAI O1 和 DeepSeek-R1 之后, 社区开始重视一种新路径: 通过 RL 让模型在输出轨迹里出现自我检查、反思、回退、验证, 也就是论文中说的 complex CoT 或 human-like cognitive processes。

但多模态模型 MLLM 面临一个额外困难:

- LLM 的输入是文本 token。
- MLLM 的输入包含图像, 图像里的几何关系、图表数值、空间位置很容易丢失。
- DeepSeek-R1 这类 text-only reasoner 推理强, 但看不到图像。
- 普通 caption 往往不包含解题所需的关键视觉事实。

所以直接把 R1-style RL 搬到 MLLM 上会遇到两个瓶颈。

第一, 缺少大规模高质量多模态推理数据。多模态数学题里, 正确推理往往依赖图像细节。只用 question 和 answer 不足以让 RL 稳定发现复杂推理。

第二, 直接 RL 的搜索空间太大。模型既要学会读图, 又要学会长推理, 还要学会格式和答案正确。reward 稀疏时, 它很容易学到短回答、假推理、或很长但错误的输出。

Vision-R1 的故事就是:

先不要期待 RL 从零自动长出多模态复杂推理。先用强 text-only reasoner 和 MLLM 合成 cold-start 数据, 把推理风格和图像信息对齐起来。然后再用 RL 优化答案正确性和推理长度。

2. 论文主张和贡献

论文提出的模型叫 Vision-R1, 属于 reasoning MLLM。它不是发明一个新视觉 encoder, 也不是只改 prompt。它的主要贡献是训练 pipeline:

- 用现有 MLLM 和 DeepSeek-R1 构造 200K Vision-R1-cold 多模态 CoT 数据。
- 用这个数据对 Qwen2.5-VL 做 SFT cold-start, 得到 Vision-R1-CI。
- 在约 10K 多模态数学题上使用 GRPO 继续训练。
- 使用 Progressive Thinking Suppression Training, 简称 PTST, 控制不同阶段的生成长度和每题采样数。
- 使用 hard formatting result reward function, 简称 HFRRF, 只有格式和答案同时正确时 reward 才是 1。

论文自己的三条贡献可以重建成下面这样:

1. 证明直接 RL 训练 MLLM 并不容易激发复杂推理, Vision-R1 选择 cold-start plus RL 的组合路线。
2. 提出 modality bridging, 无需人工标注构造 200K 多模态复杂 CoT 数据。
3. 提出 PTST, 缓解 cold-start 之后的 overthinking optimization problem。

注意这里的无需人工标注不是无成本。论文仍然要调用 Qwen2.5-VL-72B 和 DeepSeek-R1 这类强模型来生成数据。

3. 整体方法图: Fig.1 的真正含义

论文 Fig.1 左边是 pipeline, 右边是训练动态的对比。用 ASCII 图重画如下:

```
base MLLM
|
| SFT on Vision-R1-cold, 2 epochs
v
Vision-R1-CI
|
| GRPO with PTST
| stage1: 4K max tokens, 16 rollouts per question, 100 steps
| stage2: 8K max tokens, 8 rollouts per question, 100 steps
v
Vision-R1
```

论文还比较了几种失败或不足路线:

- Vision-R1-Zero: 不做 cold-start, 直接对 base MLLM 做 RL。
- Vision-R1-CI: 只做 cold-start, 不做 GRPO。
- Vision-R1-Long: cold-start 后直接允许 16K 长输出做 RL。
- Vision-R1: cold-start 后用 PTST, 先 4K 再 8K。

这四个名字非常重要。读论文表格时不要只看 final score, 要把每一行看成 对某个机制的反事实实验:

- Zero 问: 只靠 RL 能不能从零激发多模态推理。
- CI 问: 只有复杂 CoT 模仿, 没有 RL, 能不能保证正确。
- Long 问: 给更长上下文是不是自然更好。

- Ours 问: 先压缩再放松能否兼顾正确性和复杂性。

Table 3 的答案很直接:

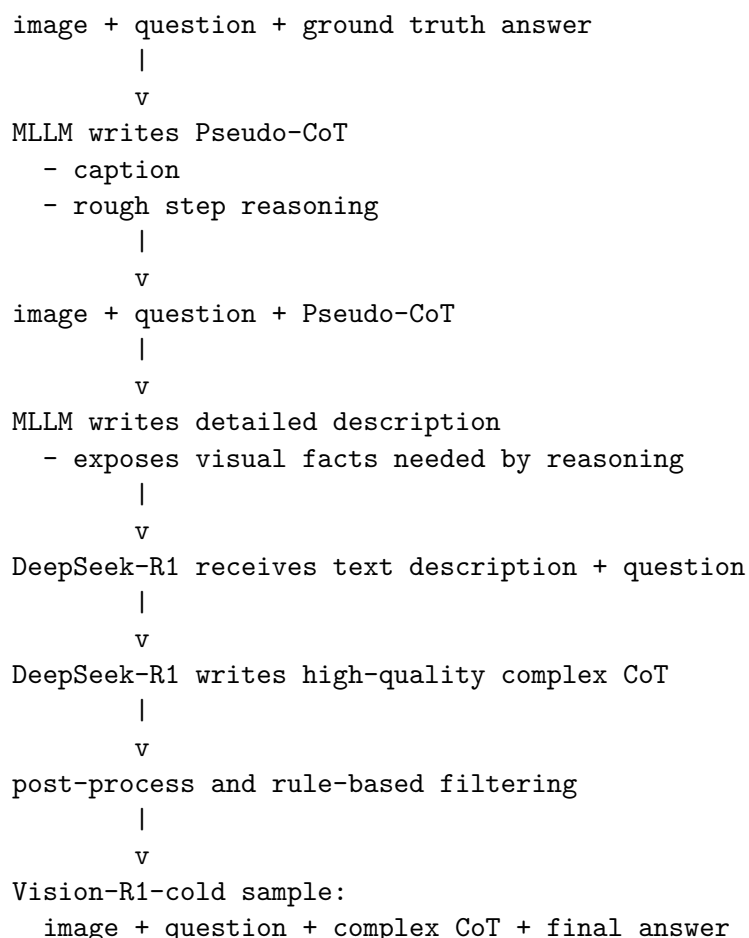
- Vision-R1-Zero 平均输出长度 1285, 平均准确率 50.7。
- Vision-R1-CI 平均输出长度 3566, 平均准确率 44.5。
- Vision-R1-Long 平均输出长度 3107, 平均准确率 47.7。
- Vision-R1 平均输出长度 2057, 平均准确率 55.4。

这说明长不等于好。cold-start 会让模型学会长推理风格, 但如果很多长链条是 错的, RL 会被错误长链条拖住。PTST 的价值是先把模型压回较短、较容易正确的轨迹, 再逐步让它变长。

4. Modality bridging: Fig.2 的数据生成机制

Vision-R1-cold 数据集不是简单把图像扔给 DeepSeek-R1。DeepSeek-R1 是 text-only LLM, 它不能直接读取图像。论文的关键桥接方法是让 MLLM 先把图像 中和解题相关的视觉事实翻译成文本。

Fig.2 的流程可以重画成:



为什么要多走一步 Pseudo-CoT?

因为普通 image caption 通常只说图里有什么, 不一定说解题需要什么。例如一张 几何图里有很多点和线, 解题关键可能是两三角形全等或某条线是垂直平分线。如果 caption 没暴露这些事实, DeepSeek-R1 只能在缺信息的文本上推理, 结果会产生 hallucination 或不确定表述。

论文 Fig.5 在 Appendix 里专门展示了这个问题。simple description 对图表数值 描述不足时, DeepSeek-R1 会说自己无法确认数据。经过 Pseudo-CoT 增强的 detailed description 包含具体数值, R1 才能给

出正确答案。

这一点是本论文最容易被低估的地方。Vision-R1 并不是让 R1 直接给图片写 CoT。它是让 MLLM 先根据题目和伪推理暴露相关视觉信息, 再让 text-only R1 写出 复杂推理。

5. Vision-R1-cold 数据集是什么

论文写道, Vision-R1-cold 最终有 200K 多模态复杂 CoT 样本。它来自已有 VQA 和多模态 CoT 数据, 但经过 modality bridging 和过滤。

主要来源和处理:

- 使用 LLaVA-CoT 数据集, 规模 100K。
- 使用 Mulberry 数据集, 规模 260K。
- 通过 Qwen2.5-VL-72B 和 DeepSeek-R1 做数据生成。
- 过滤最终答案不匹配 ground truth 的样本。
- 用规则过滤逻辑不一致样本, 并做一些语义连贯性替换。
- 最终形成 200K Vision-R1-cold。

这份数据的目的不是直接训练最终答案, 而是让 base MLLM 先学习复杂推理样式:

- question reading
- visual fact grounding
- step reasoning
- self-questioning
- reflection
- inspection
- final answer format

论文 Table 4 用反思词频来证明 Vision-R1-cold 比旧数据更像 R1 式复杂思考。数字很夸张:

- Wait: LLaVA-CoT 2300, Mulberry 1122, Vision-R1-cold 585719。
- Hmm: LLaVA-CoT 1, Mulberry 0, Vision-R1-cold 75853。
- Mistake: LLaVA-CoT 183, Mulberry 8784, Vision-R1-cold 26697。
- Alternatively: LLaVA-CoT 251, Mulberry 68, Vision-R1-cold 188187。
- Check: LLaVA-CoT 8332, Mulberry 26421, Vision-R1-cold 100148。

这组数字的含义不是词多就一定推理。它只能证明数据风格确实更接近 DeepSeek-R1 那种反思型 CoT。真正证明效果的还要看 Table 2 和后续 ablation。

6. Cold-start initialization 为什么既必要又不充分

冷启动阶段使用 Qwen2.5-VL 作为 base MLLM, 用 Vision-R1-cold 做 SFT。Appendix A 写明 SFT 做 2 epochs, 使用 Llama-Factory 框架。得到的模型叫 Vision-R1-Cl。

冷启动的必要性:

- 直接 RL 没有足够高质量多模态推理数据, 很难从零发现复杂 CoT。
- MLLM 需要先学会用视觉事实组织推理文本。
- R1 生成的数据给了模型一种怎样思考的先验。

冷启动的不充分性:

- 它是模仿学习, 不保证答案正确性提升。
- 它会让模型偏向长推理, 但长链条里可能有很多错误。

- Table 3 显示 Vision-R1-Cl 平均输出长度 3566, 但平均准确率只有 44.5, 低于 Vision-R1-Zero 的 50.7。

所以 cold-start 是把模型放到一个可优化的起点, 不是终点。它让模型会像 R1 那样写, 后面的 RL 才负责筛选和强化怎样写才更正确。

7. Overthinking optimization problem

论文提出的关键问题叫 overthinking optimization problem。它不是简单说 模型想太多, 而是一个训练优化问题。

论文观察到:

- Vision-R1-Cl 已经会输出复杂 CoT。
- 但正确的 reasoning process 往往集中在较短的 CoT 序列上。
- 如果 RL 一开始就允许 16K 输出, 模型会生成更长答案。
- 更长答案并没有带来更高性能, 还让优化更难。

可以把它理解成一个 reward landscape 的问题:

```
reasoning length
|
| short correct chains: easier to optimize, answer often right
|
| medium chains: can add verification and improve hard problems
|
| very long chains: many wrong branches, noise, self-contradiction
v
```

RL search space grows when max length is too large too early

如果一开始就给 16K, rollout 空间变大, 每题只有有限 samples。GRPO 的组内 reward 很稀疏, 训练会被很多长且错的样本污染。Vision-R1 的设计直觉是:

先让模型在短轨迹中学会正确思维, 再逐步打开更长轨迹空间。

这就是 PTST。

8. PTST: Progressive Thinking Suppression Training

PTST 的中文可以理解成渐进式思考抑制训练。它不是永远让模型短答, 而是 早期抑制过长思考, 后期逐步放宽。

论文 Fig.3 给出三阶段示意:

```
stage1: max length 4K, group size 16
stage2: max length 8K, group size 8
stage3: max length 16K, group size 4
```

但最终 Vision-R1 只采用前两阶段:

- Stage 1: 从 Vision-R1-Cl 开始, 4K token generation limit, 每题采样 16 个, 训练 100 steps。
- Stage 2: 继续训练, 8K token generation limit, 每题采样 8 个, 再训练 100 steps。
- 论文说第三阶段可以继续, 但最终 checkpoint 选在第二阶段末尾, 因为性能和 推理长度的平衡最好。

为什么 group size 会随长度变小?

每题的采样预算大致受 max length 影响。4K 时可以采样 16 个, 8K 时采样 8 个, 16K 时采样 4 个。这样每个 stage 的 sampling x length 近似控制在相似量级, 训练成本更可控。

张量级别可以这样看:

For each training batch:

```
questions:      [B]
images:         [B, C, H, W]
stage max length: L_s
stage group size: G_s
```

```
rollout tokens: [B, G_s, L_s]
logp_new:       [B, G_s]
logp_old:       [B, G_s]
logp_ref:       [B, G_s]
rewards:        [B, G_s]
advantages:     [B, G_s]
```

Stage 1 的 $G_s=16$, $L_s=4K$, Stage 2 的 $G_s=8$, $L_s=8K$ 。模型每个问题 生成一组候选, HFRRF 给每个候选打 0 或 1, GRPO 在组内标准化 reward 得到 advantage。

9. GRPO 公式怎么读

论文采用 Group Relative Policy Optimization, 即 GRPO。如果你读过 DeepSeek-R1 或 DAPO, 这里的思想会很熟:

- 对同一个问题 q , 从旧策略 π_{old} 采样 G 个输出。
- 对每个输出用 reward function 打分。
- 用组内均值和标准差把 reward 转成 advantage。
- 用 PPO-style clip 防止新策略偏离旧策略太多。
- 加 reference model KL penalty 防止策略漂移太远。

用 ASCII 写核心公式:

```
A_i = (r_i - mean(r_1 ... r_G)) / (std(r_1 ... r_G) + small)
```

```
ratio_i = pi_theta(o_i | q) / pi_old(o_i | q)
```

```
term_i = min(
    ratio_i * A_i,
    clip(ratio_i, 1 - eps, 1 + eps) * A_i
)
```

```
J = average_i(term_i - beta * KL_i)
```

论文设置:

- PPO clip 参数 $\text{eps} = 0.2$ 。
- KL 系数 $\text{beta} = 1e-2$ 。
- Advantage 来自同一 question 的 group rewards。
- Reward 是 HFRRF, 不是单独的格式 reward 和答案 reward 加权和。

这里的直觉:

- 同一题的多个答案互相比, 不需要单独训练 value model。
- 答对且格式正确的候选在组内 advantage 更高。
- clip 限制一次 update 不能过猛。
- reference KL 保留基础模型的语言分布和稳定性。

10. HFRRF: hard formatting result reward

在 RL-only 试验里, 论文 Sec.3.1 使用 formatting reward 和 result reward, 比例是 1:1。也就是格式对可以拿一部分分, 答案对可以拿另一部分分。

但在 Vision-R1 的 PTST 训练里, 论文改成 hard formatting result reward:

```
reward = 1
if output has required format and final answer is correct
```

```
reward = 0
otherwise
```

要求的格式是:

```
<think>
reasoning process
</think>
<answer>
final answer
</answer>
```

为什么要 hard?

因为 Vision-R1-CI 已经通过 cold-start 学会了强格式能力。此时继续给 格式正确但答案错的输出奖励, 会鼓励模型把精力放在包装上, 而不是正确性。HFRRF 把任务变成更硬的筛选:

- 只有会按格式回答不够。
- 只有答案可能对但格式乱也不够。
- 必须在可解析格式里给出正确答案。

本仓库对应代码在:

```
learning/multimodal-agent/src/vision_r1_original_minimal.py
```

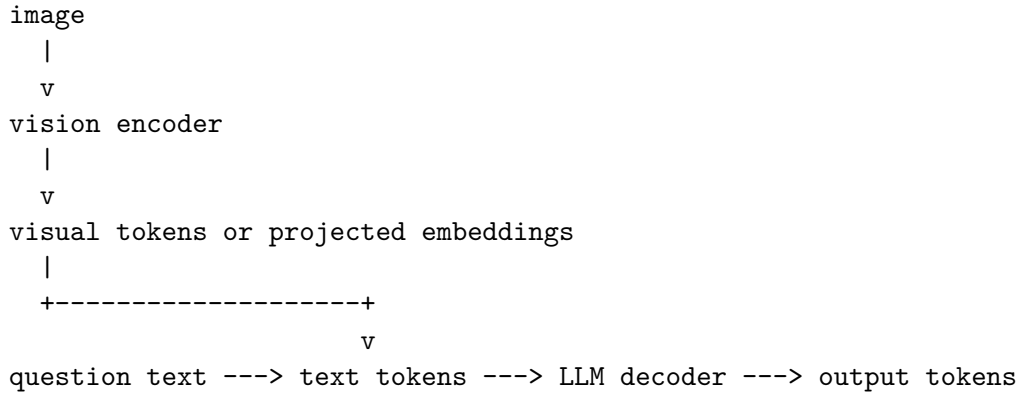
核心函数:

```
def hard_format_result_reward(response: str, gold_answer: str) -> float:
    if not has_vision_r1_format(response):
        return 0.0
    predicted = extract_final_answer(response)
    return 1.0 if predicted == gold_answer.upper() else 0.0
```

这段代码不训练真实 VLM, 但它准确保留了论文 reward 的逻辑边界。

11. 网络和信息流的张量图

真实 Qwen2.5-VL 这类 MLLM 大体可以抽象成:



在 Vision-R1 里, 训练时有两层信息流:

第一层是数据生成的信息流:

```

image facts
-> MLLM pseudo-CoT
-> MLLM detailed description
-> DeepSeek-R1 text CoT
-> paired back with image
  
```

第二层是真正训练 Vision-R1 的信息流:

```

image + question
-> Qwen2.5-VL policy
-> G_s sampled answers
-> HFRRF rewards
-> group-relative advantages
-> GRPO update with reference KL
  
```

张量形状可以按一个 batch 理解:

```

B = batch size
G = rollouts per question
L = max generated tokens in current PTST stage
V = vocabulary size
  
```

```

image tensor:      [B, C, H, W]
question tokens:   [B, T_question]
generated tokens:  [B, G, L]
token logits:      [B, G, L, V]
sequence log prob: [B, G]
reward:            [B, G]
advantage:         [B, G]
  
```

真正占显存和时间的是 $[B, G, L]$ 这部分。PTST 用更短 L 换更大 G , 再逐步 增加 L , 是一个训练预算和优化稳定性的折中。

12. Table 1: 主结果说明了什么

Table 1 比较 Vision-R1 和闭源、开源 general MLLM、math MLLM、reasoning MLLM。评测集中在 multimodal math reasoning:

- MathVista, 包含 GEO, ALG, GPS, MWP 等子项。
- MathVerse。
- MM-Math。
- DynaMath。

关键结果:

- Qwen2.5-VL-7B baseline 的 MathVista ALL 是 68.1。
- Vision-R1-7B 的 MathVista ALL 是 73.5, 提升 5.4。
- OpenAI O1 的 MathVista 是 73.9, Vision-R1-7B 只低 0.4。
- Vision-R1-7B 在 MathVerse 是 52.4, Qwen2.5-VL-7B 是 46.7。
- Vision-R1-7B 在 MM-Math 是 40.2, Qwen2.5-VL-7B 是 34.1。
- Vision-R1-7B 的平均分是 55.6, Qwen2.5-VL-7B 是 49.9。

更大模型:

- Vision-R1-32B MathVista 是 76.4, 平均分 64.9。
- Vision-R1-72B MathVista 是 78.2, 平均分 66.8。
- 论文标注 32B 和 72B 在 RL 阶段使用了额外数据。

怎么读这个表:

- 它支持 R1-style training 可以显著增强多模态数学推理。
- 它支持 7B 模型经该流程后可以接近强闭源 reasoning model 的某些榜单表现。
- 它不证明 Vision-R1 在所有多模态任务都全面超过大模型。
- 它也不证明 CoT 过程一定忠实, 因为主指标仍然是最终答案准确率。

13. Table 2: Vision-R1-cold 数据质量证据

Table 2 用 Llama-3.2-11B-V-Instruct 作为 base MLLM, 比较 cold-start 后的 Vision-R1-LlamaV-Cl-11B 和其他模型。这个表不是最终 Vision-R1 的主结果, 而是证明 Vision-R1-cold 数据集本身有价值。

关键数字:

- MMStar: 49.8 提升到 61.4。
- ChartQA: 83.4 提升到 83.9。
- MME sum: 1787 提升到 2190。
- HallBench: 40.3 提升到 49.5。
- MathVista: 48.6 提升到 62.7。
- MathVerse: 8.4 提升到 27.1。
- MM-Math: 4.1 提升到 26.1。

这说明 Vision-R1-cold 不只是让模型输出风格更长, 还显著改善了多模态数学 和部分通用多模态 benchmark。它为后续 GRPO 提供更好的起点。

14. Table 3: 三个组件缺一不可

Table 3 是理解论文最关键的消融之一。它看四种配置:

- Vision-R1-Zero: GRPO only。
- Vision-R1-Cl: cold-start only。
- Vision-R1-Long: cold-start plus GRPO, 但直接长输出。
- Vision-R1: cold-start plus GRPO plus PTST。

数字:

- Zero: 平均长度 1285, 平均准确率 50.7。
- CI: 平均长度 3566, 平均准确率 44.5。
- Long: 平均长度 3107, 平均准确率 47.7。
- Ours: 平均长度 2057, 平均准确率 55.4。

读法:

- Zero 长度短, 说明直接 RL 不容易激发复杂 CoT。
- CI 长度很长但准确率低, 说明模仿复杂 CoT 会引入 overthinking。
- Long 仍然差, 说明直接给长上下文 RL 不是解决办法。
- PTST 最好, 说明长度课程设计是关键。

这个表把论文的 story 串起来了:

cold-start 提供推理先验, RL 提供正确性选择, PTST 提供优化路径。

15. Table 5: PTST 为什么是两阶段

Table 5 比较不同 PTST stage 组合:

- Baseline 平均 49.6。
- 4Kx16 后继续 4Kx16, 平均 54.3。
- 4Kx16 后继续 8Kx16, 平均 55.3。
- 4Kx16 后 6Kx12 再 8Kx8, 平均 55.1。
- 16Kx4 后继续 16Kx4, 平均 47.7。
- 16Kx16 后继续 16Kx16, 平均 47.9。
- Vision-R1 使用 4Kx16 后 8Kx8, 平均 55.4。

结论:

- 早期固定短长度比一开始 16K 好很多。
- Stage 2 放宽到 8K 有收益。
- 继续增加 sampling 或插入额外 stage 收益不明显。
- 4Kx16 plus 8Kx8 是论文选择的简单有效配置。

这也说明 PTST 不是追求越长越好, 而是追求先正确, 再复杂。

16. Table 6: cold-start 仍然是前提

Table 6 问一个更尖锐的问题: 如果没有 Vision-R1-cold, 只给 Zero 加 PTST, 能不能解决问题?

结果:

- Vision-R1-Zero 平均 50.7。
- Zero+PTST 平均 51.8。
- Zero+SFT+PTST 平均 39.8。
- Vision-R1 平均 55.4。

含义:

- PTST alone 有一点帮助, 但不多。
- 只做没有 CoT annotation 的 SFT 反而伤害很大。
- Vision-R1-cold 的复杂 CoT 先验是 RL 能学好的关键前提。

这让论文的设计闭环成立:

modality bridging builds useful complex CoT
|
v
cold-start teaches MLLM the reasoning prior
|
v
PTST plus GRPO optimizes correctness under controlled length

17. Appendix A: 复现条件

Appendix A 给出训练和数据细节, 对你判断能否本机复现很重要。

冷启动数据:

- LLaVA-CoT 100K。
- Mulberry 260K。
- 经过 Vision-R1 的生成和过滤流程得到 200K Vision-R1-cold。

GRPO 训练数据:

- We-Math。
- MathVision。
- Polymath。
- SceMQA。
- Geometry3K。
- 总量约 10K。

32B 和 72B 额外 RL 数据:

- 将 2024 年以前的 text-based AIME 数据渲染成图像。
- 从 MAMmoth-VL 和 MMIQ 抽取子集。
- 额外约 20K 数据。

数据生成模型:

- Qwen2.5-VL-72B 用于处理 VQA 数据和生成多模态描述。
- DeepSeek-R1 用于从文本描述生成复杂 CoT。

训练框架:

- cold-start SFT 使用 Llama-Factory。
- GRPO 训练使用 Verl。

训练阶段:

- Vision-R1-Zero: 4K token length, 16 samples, 300 steps。
- Vision-R1-CI: 用 Vision-R1-cold 做 cold-start。
- Vision-R1-Long: 16K token length, 4 samples, 300 steps。
- Vision-R1: Stage 1 4Kx16 训练 100 steps, Stage 2 8Kx8 再训练 100 steps。

消融评估:

- Table 5 和 Table 6 选择最终 50 steps 中每 5 steps 评一次的最佳 checkpoint, 用来减少训练不稳定带来的统计偏差。

这意味着普通笔记本不适合完整复现训练。你应该复现机制, 不是复现大规模训练。

18. 本仓库代码怎么对应论文

本模块新增的最小机制文件:

learning/multimodal-agent/src/vision_r1_original_minimal.py

它对应论文的三个核心块:

- build_cold_start_sample: 对应 Fig.2 的 modality bridging。
- hard_format_result_reward: 对应 Sec.3.4 的 HFRRF。
- group_relative_advantages 和 grpo_clipped_surrogate_terms: 对应 Eq.1。
- PTSTStage 和 VISION_R1_PTST_SCHEDULE: 对应 Fig.3 和 Appendix A。
- score_stage_group: 对应某个 PTST stage 下先长度过滤, 再 reward 的动作。

你可以运行:

```
.venv\Scripts\python.exe learning\multimodal-agent\src\vision_r1_original_minimal.py  
.venv\Scripts\python.exe learning\multimodal-agent\src\tests\test_vlm_r1_graduation.py
```

这不会下载大模型, 也不会训练 VLM。它只验证论文机制:

- bridged description 是否保留视觉事实。
- HFRRF 是否要求格式和答案同时正确。
- GRPO advantage 是否组内均值为 0。
- PTST 是否先按 stage 长度过滤候选。

19. 代码样例: 用 toy 数据理解 modality bridging

下面是本仓库最小实现的学习用例:

```
from vision_r1_original_minimal import (  
    MultimodalQA,  
    build_cold_start_sample,  
    hard_format_result_reward,  
)  
  
item = MultimodalQA(  
    image_facts=(  
        "Triangle ABC is congruent to triangle FDE.",  
        "AF is 10 units and AD is 3.5 units.",  
        "The answer option for BD is A.",  
    ),  
    question="Find BD. Options: A 3, B 3.5, C 6, D 7.",  
    answer="A",  
)  
  
sample = build_cold_start_sample(item)  
print(sample.detailed_description)  
print(sample.response)  
print(hard_format_result_reward(sample.response, "A"))
```

你要观察的不是输出像不像真实大模型, 而是信息依赖关系:

- 没有 image_facts, R1-style reasoner 无法知道图像细节。
- Pseudo-CoT 的作用是提示哪些视觉事实和问题相关。

- detailed description 是 text-only reasoner 可以消费的桥。
- 最终 response 再和 image-question pair 配对, 成为 cold-start sample。

20. 代码样例: GRPO 和 PTST 的最小数值图

用简化 reward 看组内 advantage:

```
import torch

from vision_r1_original_minimal import group_relative_advantages

rewards = torch.tensor([1.0, 0.0, 1.0, 0.0])
advantages = group_relative_advantages(rewards)
print(advantages)
print(advantages.mean())
```

含义:

- 同一问题采样 4 个输出。
- 两个格式和答案都正确, reward 是 1。
- 两个失败, reward 是 0。
- GRPO 不需要 value model, 直接用组内相对分数。

PTST 的 toy 过滤:

```
from vision_r1_original_minimal import PTSTStage, score_stage_group

stage = PTSTStage("toy", max_tokens=5, group_size=8, steps=1)

short_good = "<think>short correct</think><answer>Final Answer:A</answer>"
long_good = "<think>" + " token" * 12 + "</think><answer>Final Answer:A</answer>"
wrong = "<think>short wrong</think><answer>Final Answer:B</answer>"

print(score_stage_group([short_good, long_good, wrong], "A", stage))
```

这个例子解释 PTST 的本质:

- 长度过长的候选在当前 stage 不进入组。
- 进入组后再由 HFRRF 打分。
- 早期 stage 不是惩罚所有长思考, 而是暂时缩小搜索空间。

21. 新手容易误读的地方

误读 1: Vision-R1 是直接 RL 训练多模态模型。

更准确: 论文证明直接 RL 不够好, 最终方案是 cold-start plus GRPO plus PTST。

误读 2: 更长 CoT 一定更强。

更准确: Table 3 和 Table 5 都说明, 太早允许长 CoT 会恶化训练。正确性先于 复杂性。

误读 3: Table 4 的反思词频证明模型真的会反思。

更准确: 它证明数据风格更接近 R1 式复杂 CoT, 但过程是否忠实仍需要更严格 评估。

误读 4: Modality bridging 只是 caption。

更准确: 它用 Pseudo-CoT 引导 MLLM 生成解题相关的 detailed description, 目的是减少从图像到文本的关键信息损失。

误读 5: HFRRF 只是格式 reward。

更准确: HFRRF 要求格式和答案同时正确。格式正确但答案错也是 0。

22. 局限性和你该保持的怀疑

论文没有声称解决所有多模态推理问题。你应该记住这些限制:

- 评测主要集中在 multimodal math reasoning, 不能直接外推到所有视觉任务。
- 训练依赖强 teacher models, 包括 Qwen2.5-VL-72B 和 DeepSeek-R1。
- 200K cold-start 数据是生成数据, 质量取决于 teacher 和过滤规则。
- CoT 中出现 Wait 或 Check 不等于推理过程一定忠实。
- HFRRF 依赖可验证答案, 对开放式视觉问答不一定容易使用。
- 长上下文训练成本高, 论文的 4K/8K/16K schedule 仍然需要大规模 GPU 环境。
- 论文主表格强调 accuracy, 对延迟、成本、鲁棒性和过程可信度讨论较少。

这些局限不是否定论文, 而是告诉你这篇论文真正贡献在哪里:

它给出了一个可复用训练范式, 而不是给出了所有多模态 agent 的最终答案。

23. 对今天多模态 agent 的意义

Vision-R1 对多模态 agent 很重要, 因为 agent 往往不是只做 caption, 而是要在视觉环境里执行多步任务:

- 看图表并计算。
- 看网页截图并定位按钮。
- 看几何图并推理。
- 看 UI 状态并规划下一步。
- 解释视觉证据和最终动作之间的关系。

这篇论文告诉你三件工程原则:

第一, 不要只把图像压成一个笼统 caption。要让模型产生和任务相关的视觉事实。

第二, 不要只模仿长 CoT。要有能验证结果的 reward 或 evaluator。

第三, 不要一开始就打开最大推理预算。训练和推理都需要预算课程。

如果你以后做 multimodal computer-use agent, Vision-R1 的思想可以迁移成:

```
screenshot
-> task-focused visual state description
-> planner reasoning
-> action candidates
-> verifier reward
-> budget-controlled policy update or selection
```

这就是它和 agent 方向的连接。

24. 怎样用 AI agent 学这篇论文, 才能真的进脑袋

你要把 agent 当成教练和检查器, 不是当成替你读完的人。

推荐流程:

1. 先自己读 Abstract, Fig.1, Fig.2, Fig.3, Table 3, Table 5。
2. 不看导读, 自己写 150 字: 为什么 Vision-R1 不直接 RL, 为什么要 cold-start 和 PTST。
3. 让 agent 按论文检查你的解释, 要求它指出哪一条 claim 没有表格证据。
4. 打开 vision_r1_original_minimal.py, 让 agent 要求你把每个函数映射到论文的 section 或 figure。
5. 自己改一个 toy stage, 例如把 max_tokens=5 改成 max_tokens=20, 预测 reward group 会怎么变, 再跑测试。
6. 让 agent 只问一个问题, 你回答后再追问。不要让它一次性给完整答案。

可直接使用的提示词:

我正在学习 Vision-R1 论文。

请你按 Fig.1 pipeline, Fig.2 modality bridging, Fig.3 PTST, Eq.1 GRPO, Table 3/5/6 evidence 的顺序考我。

一次只问一个问题。

每次我回答后, 请指出:

1. 我是否混淆了 cold-start, GRPO, PTST。
2. 我的说法对应论文哪张图或哪张表。
3. 我应该打开本仓库哪个代码函数验证。

不要直接替我总结整篇论文。

这个流程能防止我看懂了的幻觉。真正的掌握是你能闭卷解释:

- 为什么简单 caption 不够。
- 为什么 cold-start 会造成 overthinking。
- 为什么 16K 直接训比 4K 到 8K 差。
- 为什么 HFRRF 必须把 format 和 result 绑在一起。
- 为什么 Table 6 说明 PTST alone 不够。

25. 30-60 分钟本地练习

练习 A: 改 PTST toy stage。

- 打开 vision_r1_original_minimal.py。
- 找到 PTSTStage。
- 用一个很小的 max_tokens 过滤长候选。
- 预测哪些候选会进入 group。
- 跑测试并解释 reward 变化。

练习 B: 改 HFRRF。

- 让 bad_format = "Final Answer:C"。
- 预测 reward 是否应该为 0。
- 再让格式正确但答案错。
- 解释为什么两者都不能给 1。

练习 C: 重画 Fig.2。

- 不看论文, 用 6 个框画出 modality bridging。
- 必须包含 Pseudo-CoT, detailed description, DeepSeek-R1, filtering。
- 如果你只画 image -> caption -> CoT, 说明还没抓住论文贡献。

练习 D: 复述 Table 3。

- 用 4 行写 Zero, CI, Long, Ours。
- 每行写平均长度和平均准确率。
- 用一句话说明为什么 Ours 不是最长, 却最准。

26. 闭卷掌握检查

读完后你应该能回答:

- Vision-R1-Zero 为什么失败或不够强。
- Vision-R1-cold 的 200K 样本是怎样构造出来的。
- Pseudo-CoT 在 modality bridging 中有什么作用。
- 为什么 DeepSeek-R1 不能直接生成多模态 CoT。
- Vision-R1-CI 为什么会有 overthinking optimization problem。
- PTST 的 stage1 和 stage2 分别是什么配置。
- HFRRF 和 1:1 format/result reward 有什么不同。
- GRPO 的 group-relative advantage 为什么不需要 value model。
- Table 1, Table 2, Table 3, Table 5, Table 6 各自证明什么。
- 本仓库哪个文件实现了论文机制, 哪些测试验证了这些机制。

27. 最后一口气总结

Vision-R1 的核心不是让 VLM 多想一点, 而是先用 modality bridging 把图像 信息变成 text-only reasoner 能使用的详细描述, 借 DeepSeek-R1 生成高质量 复杂 CoT, 用 200K Vision-R1-cold 给 MLLM 做 cold-start, 再用 GRPO 和 PTST 在可验证多模态数学题上训练。实验的关键证据是: 直接 RL 不够, 单独 cold-start 会 overthink, 直接长上下文 RL 也不够, 只有 cold-start plus PTST 能在长度和正确性之间取得最好的平衡。